

09강

머신러닝

선형 판별 분석

통계·데이터과학과 박준희



✦ 학습목차

- 1 결정 초평면
- 2 Fisher의 선형 판별 분석
- 3 다중 클래스로 확장



1

결정 초평면



● 결정 초평면(Decision Hyperplane)

이 초평면은 일반적으로 다음과 같은 식으로 표현된다.

$$\mathbf{w}^T \mathbf{x} + w_0 = 0.$$

- $\mathbf{w} \in \mathbb{R}^D$ 는 초평면의 **법선 벡터(normal vector)**
- w_0 는 스칼라 **바이어스(bias)** 항
- $\mathbf{x}$ 는 입력 벡터(예: 2차원 혹은 D차원)

● 결정 초평면(Decision Hyperplane)

초평면 상의 임의의 두 점 $\mathbf{x}_1, \mathbf{x}_2$ 를 생각해 보자. 이들은 모두

$$\mathbf{w}^T \mathbf{x}_1 + w_0 = 0, \quad \mathbf{w}^T \mathbf{x}_2 + w_0 = 0$$

를 만족한다. 두 식을 빼면,

$$\mathbf{w}^T (\mathbf{x}_2 - \mathbf{x}_1) = 0.$$

● 결정 초평면(Decision Hyperplane)

이제 초평면 $\mathbf{w}^T \mathbf{x} + w_0 = 0$ 와 임의의 점 $\mathbf{x}^*$ 사이의 **최단 거리**를 계산해 보자.
최단 거리를 d 라고 할 때,

$$d = \frac{|\mathbf{w}^T \mathbf{x}^* + w_0|}{\|\mathbf{w}\|}.$$

참고: 점과 직선 사이의 거리

점 (x_1, y_1) 과 직선 $ax + by + c = 0$ 사이의 거리 d 는 다음과 같다:

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}.$$

1

결정 초평면

● 결정 초평면(Decision Hyperplane)

```
[1]: import numpy as np
import matplotlib.pyplot as plt

[2]: # -----
# 1) 결정 초평면(직선)  $w^T x + w_0 = 0$  정의
#     예:  $w = [1, 2]$ ,  $w_0 = -3 \Rightarrow x + 2y - 3 = 0$ 
# -----
w = np.array([1, 2]) # 법선 벡터 w
w0 = -3              # 바이어스(상수항) w0

# -----
# 2) 임의의 점  $x^*$  정의
# -----
x_star = np.array([2, 3]) # 예: (2, 3)
```

● 결정 초평면(Decision Hyperplane)

```
# -----
# 3) 점-초평면(직선) 사이의 거리 계산
#  $d = |w^T x^* + w_0| / ||w||$ 
# -----
dist = abs(w.dot(x_star) + w0) / np.linalg.norm(w)
print("결정 초평면과 점", x_star, "사이의 거리 =", dist)

# -----
# 4) 직선을 그리기 위한  $y = (-w_0 - w[0]*x) / w[1]$  형태로 변환
#  $w[0]*x + w[1]*y + w_0 = 0$ 
#  $\Rightarrow y = - (w[0]/w[1]) * x - (w_0 / w[1])$ 
# -----
def line_equation(x, w, w0):
    return -(w[0]/w[1])*x - (w0/w[1])
```

● 결정 초평면(Decision Hyperplane)

```
# 직선을 그릴 x 범위 설정
x_vals = np.linspace(-2, 6, 100)
y_vals = line_equation(x_vals, w, w0)

# -----
# 5) 점 x_star에서 직선으로의 수직 내린 점(수선의 발) 좌표 구하기
#
# 결정 초평면:  $w^T x + w_0 = 0$ 
# 점: x_star
# 수선의 발: x_foot
#
# x_foot는 다음을 만족:
# 1)  $w^T x_{foot} + w_0 = 0$  (직선 위)
# 2)  $x_{foot} - x_{star}$ 가 w와 평행 (법선 방향)
#
# →  $x_{foot} = x_{star} - ((w^T x_{star} + w_0) / ||w||^2) * w$ 
# -----
w_norm_sq = w.dot(w) #  $||w||^2$ 
x_foot = x_star - ((w.dot(x_star) + w0) / w_norm_sq) * w
```

출처:파이썬 프로그램 실습내용 중

● 결정 초평면(Decision Hyperplane)

```
plt.figure(figsize=(6,6))

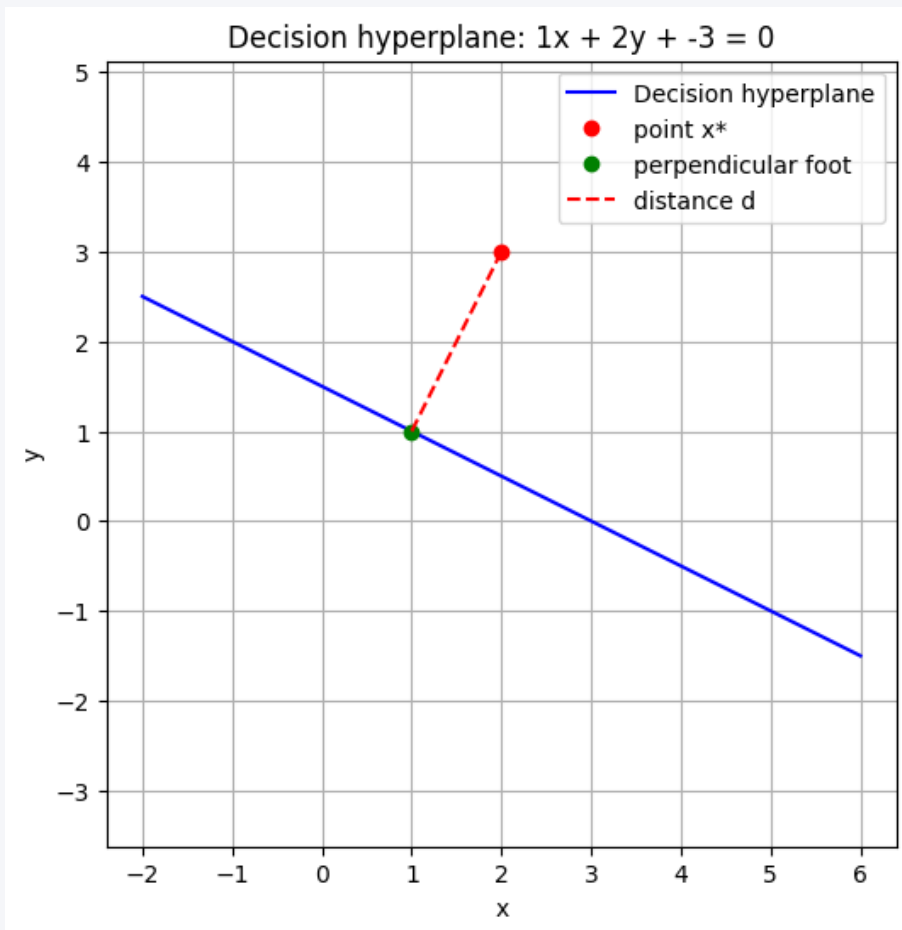
# 결정 초평면( 직선) 그리기
plt.plot(x_vals, y_vals, 'b-', label='Decision hyperplane')

# 임의의 점 x_star 표시
plt.plot(x_star[0], x_star[1], 'ro', label='point x*')

# 수선의 발 x_foot 표시 및 연결선 그리기
plt.plot(x_foot[0], x_foot[1], 'go', label='perpendicular foot')
plt.plot([x_star[0], x_foot[0]], [x_star[1], x_foot[1]], 'r--', label='distance d')

# 그래프 설정
plt.title(f"Decision hyperplane: {w[0]}x + {w[1]}y + {w0} = 0")
plt.xlabel("x")
plt.ylabel("y")
plt.axis('equal')
plt.grid(True)
plt.legend()
plt.show()
```

● 결정 초평면(Decision Hyperplane)



출처:파이썬 프로그램 실습내용 중

2

피셔의 선형판별분석



2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

이진 분류(binary classification): 클래스가 두 개라 가정하고, 각 클래스를 C_1, C_2 라고 해보자.

LDA는 두 개의 클래스가 주어졌을 때, 고차원 공간에서 **한 줄(1차원)**로 데이터를 투영시켜 각 클래스를 가장 잘 구분되도록 만드는 방법이다.

- 각 클래스에 속하는 데이터 포인트:

$$\{\mathbf{x}_n \mid \mathbf{x}_n \in C_i\}, \quad i \in \{1, 2\}.$$

- 각 클래스의 평균 벡터(평균 좌표):

$$\mathbf{m}_1 = \frac{1}{N_1} \sum_{n \in C_1} \mathbf{x}_n, \quad \mathbf{m}_2 = \frac{1}{N_2} \sum_{n \in C_2} \mathbf{x}_n.$$

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

우리는 데이터 $\mathbf{x}$ 를 1차원 축 $\mathbf{w}$ 방향으로 사영(projection)해서,

$$y = \mathbf{w}^T \mathbf{x}$$

로 변환하고 싶다. 그런데 이 때,

1. 클래스 1과 클래스 2의 투영된 평균값 m_1, m_2 이 서로 최대한 멀어지도록(즉 $(m_2 - m_1)^2$ 가 커지도록)
2. 클래스 1 내부 데이터의 변동성과 클래스 2 내부 데이터의 변동성을 더한 값($N_1 s_1^2 + N_2 s_2^2$)은 작아지도록

하고 싶다.

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

이를 하나의 성능 지표(Performance Index)로 표현하면,

$$PI_{LDA}(\mathbf{w}) = \frac{(m_2 - m_1)^2}{N_1 s_1^2 + N_2 s_2^2}.$$

여기서,

- $m_i = \mathbf{w}^T \mathbf{m}_i$: 클래스 i 의 평균 $\mathbf{m}_i$ 를 $\mathbf{w}$ 에 사영한 값.
- s_i^2 : 클래스 i 에 속한 데이터들을 $\mathbf{w}$ 에 사영했을 때의 표본분산.
 $s_i^2 = \frac{1}{N_i} \sum_{n \in C_i} (y_n - m_i)^2$. 여기서, $y_n = \mathbf{w}^T \mathbf{x}_n$.
- N_i : 클래스 i 의 데이터 개수.

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

성능 지표 $PI_{LDA}(\mathbf{w})$ 의 분자 부분을 보자.

$$\begin{aligned}(m_2 - m_1)^2 &= (\mathbf{w}^T(\mathbf{m}_2 - \mathbf{m}_1))^2 \\ &= \mathbf{w}^T(\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T \mathbf{w}.\end{aligned}$$

여기서, 클래스 간 산포 행렬 $S_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T$ 로 정의하여, 위 식을

$$\mathbf{w}^T S_B \mathbf{w}.$$

로 표현할 수 있다.

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

성능 지표 $PI_{LDA}(\mathbf{w})$ 의 분모 부분을 보자.

$$\begin{aligned}
 N_1 s_1^2 + N_2 s_2^2 &= \sum_{i=1}^2 \sum_{n \in C_i} (y_n - m_i)^2 \\
 &= \sum_{i=1}^2 \sum_{n \in C_i} (\mathbf{w}^T \mathbf{x}_n - \mathbf{w}^T \mathbf{m}_i)^2 \\
 &= \sum_{i=1}^2 \sum_{n \in C_i} \mathbf{w}^T (\mathbf{x}_n - \mathbf{m}_i) (\mathbf{x}_n - \mathbf{m}_i)^T \mathbf{w}.
 \end{aligned}$$

여기서, 클래스 내 산포 행렬 $S_W = \sum_{i=1}^2 \sum_{n \in C_i} (\mathbf{x}_n - \mathbf{m}_i) (\mathbf{x}_n - \mathbf{m}_i)^T$ 로 정의하여, 위 식을

$$\mathbf{w}^T S_W \mathbf{w}.$$

로 표현할 수 있다.

출처:파이썬 프로그램 실습내용 중

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

$$PI_{LDA}(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}.$$

이를 최대화하는 $\mathbf{w}$ 를 찾고 싶다.

최적 $\mathbf{w}_{LDA}$:

위 식을 $\mathbf{w}$ 에 대해 벡터미분하고 0벡터가 나오는(즉, PI_{LDA} 를 최대화하는) $\mathbf{w}$ 를 구하면 다음과 같다.

$$\mathbf{w}_{LDA} \propto S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1).$$

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

LDA에서 우리가 최적화하려는 목적함수(성능 지표)는

$$PI_{LDA}(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

이다. 이와 같은 형태는 **Rayleigh quotient**(레이리 몫)이라 불리며, 다음과 같은 중요한 성질을 갖는다.

- $\mathbf{w}$ 에 임의의 상수 c 를 곱해도 함수값이 변하지 않는다.

$$PI_{LDA}(c\mathbf{w}) = \frac{(c\mathbf{w})^T S_B (c\mathbf{w})}{(c\mathbf{w})^T S_W (c\mathbf{w})} = \frac{c^2(\mathbf{w}^T S_B \mathbf{w})}{c^2(\mathbf{w}^T S_W \mathbf{w})} = PI_{LDA}(\mathbf{w}).$$

따라서 크기는 중요치 않고, $\mathbf{w}$ 의 방향(direction)만을 결정하면 된다.

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

라그랑주 승수를 이용한 최적화 (Lagrangian):

$\mathbf{w}^T S_W \mathbf{w} \neq 0$ 라는 제약(분모가 0이 되면 안 됨)을 고려하며 최적화를 수행하려면, 라그랑주 승수법(Lagrange multiplier method)을 사용할 수 있다.

목적함수를 직접 최적화하기보다는, 제약 조건 $\mathbf{w}^T S_W \mathbf{w} = 1$ (또는 임의의 상수)로 둔 상태에서, 다음과 같은 라그랑지언(Lagrangian)을 정의한다.

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_B \mathbf{w} - \lambda(\mathbf{w}^T S_W \mathbf{w} - 1).$$

여기서 λ 는 라그랑주 승수이다. 이 함수를 $\mathbf{w}$ 에 대해 미분하여 0벡터가 되도록 하는 $\mathbf{w}$ 를 찾으면, 우리가 원하는 최적해(또는 그 방향)를 찾을 수 있다.

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

라그랑지언 미분 및 정리:

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T S_B \mathbf{w} - \lambda (\mathbf{w}^T S_W \mathbf{w} - 1).$$

이를 $\mathbf{w}$ 로 미분하면,

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^T S_B \mathbf{w}) - \lambda \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^T S_W \mathbf{w} - 1) \\ &= 2S_B \mathbf{w} - \lambda 2S_W \mathbf{w} \\ &= \mathbf{0}. \end{aligned}$$

즉,

$$S_B \mathbf{w} = \lambda S_W \mathbf{w}.$$

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

$$S_B \mathbf{w} = (\mathbf{m}_2 - \mathbf{m}_1) \underbrace{(\mathbf{m}_2 - \mathbf{m}_1)^T \mathbf{w}}_{\text{스칼라}} = \left((\mathbf{m}_2 - \mathbf{m}_1)^T \mathbf{w} \right) (\mathbf{m}_2 - \mathbf{m}_1).$$

즉, $S_B \mathbf{w}$ 는 $(\mathbf{m}_2 - \mathbf{m}_1)$ 방향 벡터가 된다.

$$S_B \mathbf{w} = \lambda S_W \mathbf{w} \implies (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T \mathbf{w} = \lambda S_W \mathbf{w}.$$

좌변은 $(\mathbf{m}_2 - \mathbf{m}_1)$ 방향 벡터이므로, 자연스럽게 $\mathbf{w}$ 는

$$\mathbf{w} \propto S_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$$

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

```
[3]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)

# 두 개의 서로 다른 이변량 정규분포 샘플링
mean1 = [-2, -1]
cov1 = [[1.0, 0.3],
        [0.3, 1.0]] # 임의의 공분산 행렬

mean2 = [2, 3]
cov2 = [[1.0, -0.2],
        [-0.2, 1.0]]
```

2

피셔의 선형판별분석

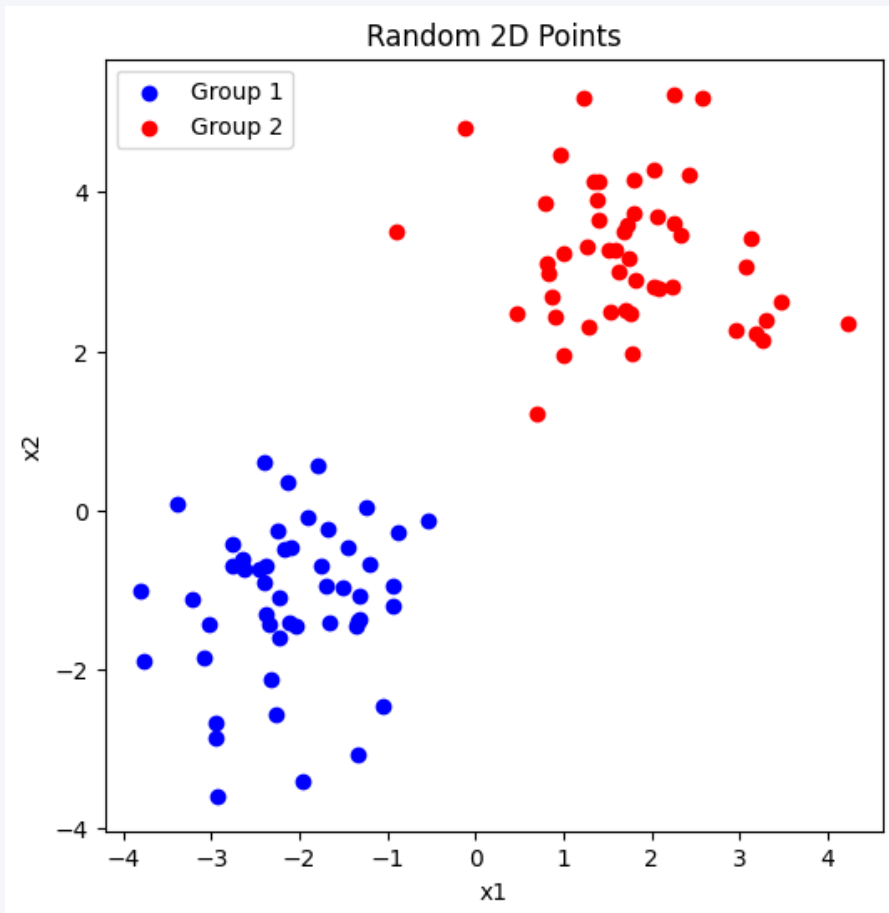
● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

```
N = 50 # 각 그룹별 점 개수
X1 = np.random.multivariate_normal(mean1, cov1, N)
X2 = np.random.multivariate_normal(mean2, cov2, N)

# 간단한 산점도 시각화
plt.figure(figsize=(6,6))
plt.scatter(X1[:,0], X1[:,1], c='blue', label='Group 1')
plt.scatter(X2[:,0], X2[:,1], c='red', label='Group 2')
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Random 2D Points")
plt.legend()
plt.show()
```

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)



출처:파이썬 프로그램 실습내용 중

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

```
[4]: m1 = np.mean(X1, axis=0) # (2,) vector
      m2 = np.mean(X2, axis=0) # (2,) vector

      print("m1 =", m1)
      print("m2 =", m2)

      # 클래스 내 산포
      S1 = np.zeros((2,2))
      for x in X1:
          diff = (x - m1).reshape(-1,1)
          S1 += diff @ diff.T

      S2 = np.zeros((2,2))
      for x in X2:
          diff = (x - m2).reshape(-1,1)
          S2 += diff @ diff.T
```

출처:파이썬 프로그램 실습내용 중

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

```
S_W = S1 + S2
print("\nS_W =\n", S_W)

# 클래스 간 산포
m_diff = (m2 - m1).reshape(-1,1)
S_B = m_diff @ m_diff.T
print("\nS_B =\n", S_B)
```

```
m1 = [-2.10123557 -1.1207998 ]
m2 = [1.7491915  3.23813303]
```

```
S_W =
[[72.68578613 -0.63064329]
 [-0.63064329 84.90896851]]
```

```
S_B =
[[14.82578867 16.78375298]
 [16.78375298 19.00029538]]
```

```
[5]: S_W_inv = np.linalg.inv(S_W)
w_LDA = S_W_inv @ (m2 - m1)
print("w_LDA =", w_LDA)
```

```
w_LDA = [0.05342244 0.05173333]
```

2

피셔의 선형판별분석

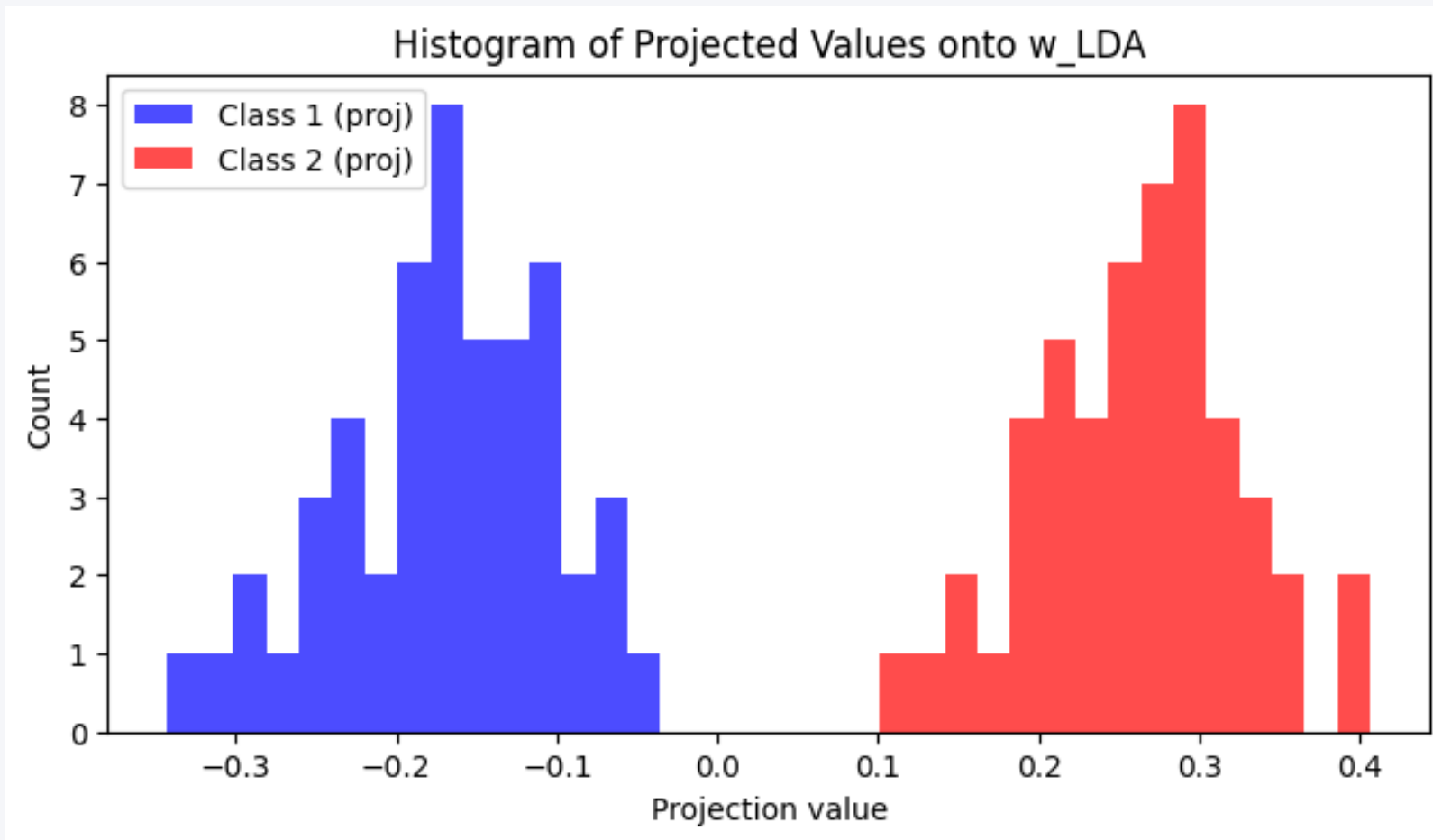
● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)

```
[7]: y1 = X1 @ w_LDA
      y2 = X2 @ w_LDA

      plt.figure(figsize=(8,4))
      plt.hist(y1, bins=15, alpha=0.7, color='blue', label='Class 1 (proj)')
      plt.hist(y2, bins=15, alpha=0.7, color='red', label='Class 2 (proj)')
      plt.title("Histogram of Projected Values onto w_LDA")
      plt.xlabel("Projection value")
      plt.ylabel("Count")
      plt.legend()
      plt.show()
```

2 피셔의 선형판별분석

● 피셔의 선형판별분석(Linear Discriminant Analysis, LDA)



출처: 파이썬 프로그램 실습내용 중

3

다중클래스로 확장



3

다중클래스로 확장

● 다중클래스 LDA

- 각 클래스별로

- 클래스 평균 벡터 $(\mathbf{m}_k = \frac{1}{N_k} \sum_{n \in C_k} \mathbf{x}_n)$
- 클래스 내부 공분산 행렬 $(\mathbf{S}_k = \sum_{n \in C_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^T)$ 등을 정의한다.

- 전체 데이터에 대해서도

- 전체 평균 $(\mathbf{m} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n = \frac{1}{N} \sum_{k=1}^K N_k \mathbf{m}_k),$
- 전체 공분산 $(\mathbf{S}_T = \sum_{n=1}^N (\mathbf{x}_n - \mathbf{m})(\mathbf{x}_n - \mathbf{m})^T)$ 등을 구해둔 뒤, 이를 바탕으로
- 클래스 간 산포 행렬(between-class scatter) $(\mathbf{S}_B = \sum_{k=1}^K N_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^T)$
- 클래스 내부 산포 행렬(within-class scatter) $(\mathbf{S}_W = \sum_{k=1}^K \mathbf{S}_k)$ 를 유도한다

출처:파이썬 프로그램 실습내용 중

3

다중클래스로 확장

● 다중클래스 LDA

- 2-클래스 LDA(피셔 판별)에서와 마찬가지로,

$$\max_{\mathbf{W}} \frac{\text{trace}(\mathbf{W}^T \mathbf{S}_B \mathbf{W})}{\text{trace}(\mathbf{W}^T \mathbf{S}_W \mathbf{W})}$$

의 형태(또는 유사한 Rayleigh quotient 형태)의 기준 함수를 세워, "클래스 간 산포 대비 클래스 내부 산포를 최대로 하는" 선형 변환 $\mathbf{W}$ 를 찾는다 ($\mathbf{W}$ 는 행렬).

3

다중클래스로 확장

● 다중클래스 LDA

1. 고유벡터(eigenvector) 계산

- 보통 " $\mathbf{S}_W^{-1} \mathbf{S}_B$ " 연산을 수행하여,

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$$

를 만족하는 고유벡터 $\mathbf{w}$ 와 고유값 λ 를 구한다.

- 이 중 고유값 λ 가 큰 방향($\mathbf{w}$)이 "클래스 간 산포는 크게, 클래스 내 산포는 작게 만들어주는" 방향이라고 해석할 수 있다.

3

다중클래스로 확장

● 다중클래스 LDA

2. 투영행렬 $\mathbf{W}$ 구성

- 위에서 구한 고유벡터들을 열(column)로 모아서 행렬 $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_m]$ 를 구성한다.
- 보통은 고유값이 가장 큰 순서대로 $K - 1$ 개(또는 더 적은 수)를 선택하여 열벡터를 채우는 방식을 사용.
- 새로운 입력 $\mathbf{x}$ 에 대해서는 $\mathbf{z} = \mathbf{W}^T \mathbf{x}$ 로 보다 낮은 차원의 공간으로 투영하게 된다.

★ 정리하기

■ 결정 초평면

점과 직선의 거리의 일반화 / 선형 분류를 위한 핵심 기준

■ 피셔의 선형 판별 분석 (이진 클래스 LDA)

클래스 간 산포를 최대화 / 클래스 내부 산포를 최소화
위 기준으로 데이터를 투영하여 차원 축소

■ 다중 클래스 LDA

고유벡터와 고유값을 추출하여 투영 행렬을 구성

10 강

다음시간안내

로지스틱 회귀

